

打刻編集申請画面の表示と機能の追加修正

現状の「未来の日付まで編集できてしまう状態」から、日付の文脈（過去・当日・未来）に合わせてスマートに挙動を変化させるべく、ロジックの考察とソースコード修正を進めていきましょう。

実装に向けたロジックの整理

まずは `AttendanceEditView.vue` に盛り込む仕様の内容を整理します。

- 翌日以降（未来の日付）の制御:

- `recordDate` が今日より大きい（未来）なら、ボタン自体を `v-if` で完全に非表示にします。

- 当日の制御:

- Entry（出勤）と備考が入力されていれば、Exit（退勤）が空欄でも申請を許可します。
- すでに一度申請してあっても、後から退勤時刻だけを追加して「再申請」できるようにフォームの入力を柔軟に受け付けます。

- 前日以前（過去の日付）の制御:

- Entry、Exit、備考の3つすべてが埋まっていることを、申請ボタンの活性化条件（または送信時チェック）とします。

- 申請取消機能（当日・過去共通）：

- 状態が「申請中（未承認）」の場合のみ、「取消」ボタンを表示します。
- 取り消されたら、バックエンドに通知してフラグを `0`（通常状態）に戻します。

フロントエンド： `AttendanceEditView.vue` の修正コード

既存の `AttendanceEditView.vue` の `<template>`（ボタン周辺・フォーム内）と `<script setup>` のロジックを以下のようにアップデートしてください。

① `<template>` 部分の修正（テーブルのボタン・フォーム周辺）

カレンダーリストの「操作」列のボタン表示を、条件分岐（`v-if` / `v-else-if`）を使って制御します。

```
<td>  
  <span v-if="isFutureDate(day.recordDate)" class="text-muted">不可</span>
```

```
<div v-else class="action-buttons-gap">
  <button @click="openEditForm(day)" class="btn-action-edit">
    {{ day.isTimechangeDemand === 1 ? '再申請' : '編集申請' }}
  </button>

  <button
    v-if="day.isTimechangeDemand === 1 && day.timechangeStatus === 0"
    @click="cancelRequest(day)"
    class="btn-action-cancel"
  >
    取消
  </button>
</div>
</td>
```

② <script setup> 部分の修正・追加

日付の比較ロジックと、送信時の厳格なバリデーション、そして「取消処理」のAPI通信を実装します。

```
// =====  
// 日付判定用のユーティリティ関数群  
// =====  
  
// 当日であるかを判定する  
const isToday = (dateStr) => {  
  const todayStr = new Date().toISOString().split('T')[0];  
  return dateStr === todayStr;  
};  
  
// 翌日以降（未来）であるかを判定する  
const isFutureDate = (dateStr) => {  
  const todayStr = new Date().toISOString().split('T')[0];  
  return dateStr > todayStr; // 文字列比較で yyyy-MM-dd の前後が判定できます  
};
```

```
// =====
// 修正・追加：申請提出時のバリデーションロジック
// =====
const submitRequest = async () => {
  const targetDate = selectedDay.value.recordDate;

  // 1. 【共通】備考欄（修正理由）の空チェック
  if (!form.value.reason.trim()) {
    alert('修正理由（備考）を入力してください。');
    return;
  }

  // 2. 【共通】時刻の簡易形式チェック（ハイフン、または hh:mm:ss 形式）
  const timeRegex = "^[01][0-9]|2[0-3]):[0-5][0-9]:[0-5][0-9]$";

  // 3. 【日付の文脈に応じた厳格なバリデーション】
  if (isToday(targetDate)) {
    // 当日の場合：Entry時刻は必須、Exitは空（またはハイフン）でも許可
    if (!form.value.entryTime.match(timeRegex)) {
      alert('当日の申請には、正しい出勤時刻(Entry)の入力が必要です。');
      return;
    }

    if (form.value.exitTime.trim() == '-') form.value.exitTime = '-';
    if (form.value.exitTime !== '-' && !form.value.exitTime.match(timeRegex)) {
      alert('退勤時刻(Exit)の形式が不正です。');
      return;
    }
  }
}
```

```
} else {  
  // 前日以前（過去）の場合：Entry、Exit両方の入力が必要  
  if (!form.value.entryTime.match(timeRegex) || !form.value.exitTime.match(timeRegex)) {  
    alert('前日以前の申請を行うには、出勤(Entry)と退勤(Exit)の両方の時刻を入力してください。');  
    return;  
  }  
}  
  
// 確認アラートを表示  
if (!confirm('この内容で打刻内容の編集申請を提出してもよろしいですか？')) {  
  return;  
}  
  
try {  
  await apiClient.post('/attendance/request-edit', {  
    accountId: loggedInAccountId.value,  
    recordDate: targetDate,  
    targetEntryTime: form.value.entryTime,  
    targetExitTime: form.value.exitTime,  
    reason: form.value.reason  
  });  
}
```

```

    alert('申請が正常に提出されました！');
    closeForm();
    fetchMonthlyData();
  } catch (error) {
    console.error('申請エラー:', error);
    alert(error.response?.data || '申請の提出に失敗しました。');
  }
};

// =====
// 【新規追加】申請取り消し処理
// =====
const cancelRequest = async (day) => {
  if (!confirm(`${formatDate(day.recordDate)} の編集申請を取り消しますか？`)) {
    return;
  }

  try {
    // 後述するバックエンドの取消専用APIへ送信
    await apiClient.post('/attendance/cancel-edit', {
      accountId: loggedInAccountId.value,
      recordDate: day.recordDate
    });

    alert('申請を取り消しました。');
    fetchMonthlyData(); // リストを再読込
  } catch (error) {
    console.error('取消エラー:', error);
    alert('申請の取り消しに失敗しました。');
  }
};

```


③ `<style scoped>` へのデザイン追加（ボタン用CSS）

取消ボタンと、横並びにするためのスタイルを少し追加します。

```
/* ボタンを横並びにするための隙間設定 */
.action-buttons-gap {
  display: flex;
  justify-content: center;
  gap: 8px;
}

/* 取り消しボタンのデザイン */
.btn-action-cancel {
  background-color: #ffffff;
  border: 1px solid #e53935;
  color: #e53935;
  padding: 6px 12px;
  border-radius: 4px;
  cursor: pointer;
  font-weight: bold;
}
```

```
    transition: all 0.2s;
}
.btn-action-cancel:hover {
    background-color: #ffebee;
}
```

3. バックエンド（Java）側：「取消機能」APIの追加

ユーザーが「取消」を押したときに、データベースの申請中フラグ（`isTimechangeDemand`）を安全に `0` に戻すための窓口をバックエンドに新設します。

① `EntryExitCalendarService.java` へのメソッド追加

```
/**
 * ユーザー自身による編集申請の取り消し処理
 */
@Transactional
public void cancelTimeChangeRequest(String accountId, LocalDate recordDate) {
    Optional<EntryExitCalendar> opt = calendarRepository.findByRegisteredAccountIdAndRecordDate(accountId, recordDate);
```

```

    if (opt.isPresent()) {
        EntryExitCalendar record = opt.get();
        // 申請中（未承認）の場合のみ取り消しを許可する安全ガード
        if (record.getIsTimechangeDemand() == 1 && record.getTimechangeStatus() == 0) {
            record.setIsTimechangeDemand(0); // フラグを通常に戻す
            record.setTmpEntryTime(null);    // 申請中だった仮時刻をクリア
            record.setTmpExitTime(null);
            record.setReason(null);

            calendarRepository.save(record);
            System.out.println("=== [Service] ユーザーにより申請が取り消されました ===");
        }
    }
}

```

② AttendanceEditController.java へのエンドポイント追加

```

/**
 * 窓口3: ユーザーからの打刻修正申請の「取り消し」リクエストを受け付ける
 * POST http://localhost:8080/api/attendance/cancel-edit
 */

```

```
@PostMapping("/cancel-edit")
public ResponseEntity<String> cancelEdit(@RequestBody Map<String, Object> payload) {
    String accountId = (String) payload.get("accountId");
    String dateStr = (String) payload.get("recordDate");

    if (accountId == null || dateStr == null) {
        return ResponseEntity.badRequest().body("必要なパラメータが不足しています。");
    }

    try {
        java.time.LocalDate recordDate = java.time.LocalDate.parse(dateStr);
        calendarService.cancelTimeChangeRequest(accountId, recordDate);
        return ResponseEntity.ok("申請を取り消しました。");
    } catch (Exception e) {
        return ResponseEntity.internalServerError().body("取り消し処理中にエラーが発生しました: " + e.getMessage());
    }
}
```

連動の確認ポイント

ソースコードを適用後、ブラウザで以下の挙動を確認してみてください。

1. **翌日の行:** ボタンが消え、「不可」などの落ち着いたテキストが表示されます。
2. **当日の行:** 「編集申請」を押し、退勤(Exit)を「 - 」のままにしても、出勤時間と備考があれば正常に「申請中」バッジが灯ります。その後、さらに退勤時間を書き足して「再申請」することも可能になります。
3. **過去の日:** 出勤または退勤のどちらかを空欄（またはハイフン）にして申請しようとする、フロントエンドの防壁が働き「両方の時刻を入力してください」と親切に弾かれます。
4. **取消ボタン:** 申請中の日の横にだけ「取消」ボタンが出現し、押すとバッジがパッと消えて元の表示に戻ります。

バックエンド側で必要な修正

ここまでにフロントエンド側ファイル `AttendanceEditView.vue` を修正してきましたが、実はこれだけではまだ当日分の打刻申請をする場合に問題が生じてしまいます。

API通信によりフロントエンドからのリクエストをバックエンドと繋げた際に、今のままでは、当日の退勤時刻（Exit）が空（またはハイフン）のときにバリデーションエラーになってしまう状態なのです。

これは現在のバックエンド（`AttendanceEditController.java`）の防壁（バリデーション）が、「**すべての日付において `hh:mm:ss` の形式でなければならない（空文字やハイフンは弾く）**」という一律のルールになってしまっていることが原因です。

この問題を解消し、仕様書通りの「当日に限り、退勤時刻が未入力（またはハイフン）でも処理を通す」ための、バックエンド側のスマートな修正・変更内容をご案内致します。今回はRepositoryの変更は不要で、**ControllerとServiceの2箇所を優しくアップデート**するだけで綺麗に解決できます。

Controller層の修正 (AttendanceEditController.java)

Controllerで行っている二重バリデーションのロジックに、「対象日が当日、かつ、退勤時刻が未入力（空またはハイフン）の場合はチェックをスキップして許容する」という条件分岐を追加します。

AttendanceEditController.java の @PostMapping("/request-edit") メソッド内を、以下のように書き換えてください。

```
/**
 * 窓口2: ユーザーからの打刻修正（編集）申請リクエストを受け付ける
 * POST http://localhost:8080/api/attendance/request-edit
 */
@PostMapping("/request-edit")
public ResponseEntity<String> requestEdit(@RequestBody com.appspace.backend.dto.TimeChangeRequest request) {
    // 【二重バリデーション】仕様書要件に基づくバックエンドチェック

    // 1. 備考（修正理由）の空っぽチェック
    if (request.getReason() == null || request.getReason().trim().isEmpty()) {
        return ResponseEntity.badRequest().body("【却下】修正理由（備考）の入力は必須です。");
    }
}
```

```
// 2. 日付判定の準備（今日の日付を取得）
java.time.LocalDate today = java.time.LocalDate.now();
java.time.LocalDate targetDate = request.getRecordDate();
boolean isToday = targetDate.equals(today);

// 3. 時刻形式チェック用の正規表現（hh:mm:ss）
String timeRegex = "^[01][0-9]|2[0-3]):[0-5][0-9]:[0-5][0-9]$";

// --- Entry（出勤）時刻のチェック ---
// 当日・過去問わず、Entry時刻は必須とします（未入力・空欄・ハイフンは不可）
if (request.getTargetEntryTime() == null ||
    request.getTargetEntryTime().equals("-") ||
    request.getTargetEntryTime().trim().isEmpty()) {
    return ResponseEntity.badRequest().body("【却下】出勤時刻(Entry)を入力してください。");
}
if (!request.getTargetEntryTime().matches(timeRegex)) {
    return ResponseEntity.badRequest().body("【却下】Entry時刻の形式が不正です。(hh:mm:ssで入力してください)");
}

// --- Exit（退勤）時刻のチェック ---
String exitTime = request.getTargetExitTime();

// 入力値が空欄、もしくはフロントからハイフン「-」で送られてきた場合、扱いやすいように「-」に統一します
if (exitTime == null || exitTime.trim().isEmpty()) {
    exitTime = "-";
}
```



```

if (isToday) {
    // 当日の場合: 退勤時刻が「-」であっても許容（出勤中につき退勤打刻がまだ無いため）
    if (!exitTime.equals("-") && !exitTime.matches(timeRegex)) {
        return ResponseEntity.badRequest().body("【却下】当日の退勤時刻(Exit)の形式が不正です。");
    }
} else {
    // 前日以前（過去）の場合: 退勤時刻「-」は不可。必ず有効な時刻形式が必要
    if (exitTime.equals("-")) {
        return ResponseEntity.badRequest().body("【却下】過去の日付の申請には、退勤時刻(Exit)の入力も必須です。");
    }
    if (!exitTime.matches(timeRegex)) {
        return ResponseEntity.badRequest().body("【却下】Exit時刻の形式が不正です。(hh:mm:ssで入力してください)");
    }
}

try {
    // 統一した変数 (exitTime) をサービス層に手渡すように修正
    calendarService.requestTimeChange(
        request.getAccountId(),
        request.getRecordDate(),
        request.getTargetEntryTime(),
        exitTime, // 「-」 または正しい時刻文字列
        request.getReason()
    );

    System.out.println("=== [Controller] 修正申請の受付に成功しました ===");
    return ResponseEntity.ok("打刻内容の修正申請を提出しました。管理者の承認をお待ちください。");
} catch (Exception e) {
    return ResponseEntity.internalServerError().body("申請処理中に重大なエラーが発生しました: " + e.getMessage());
}
}

```

Service層の補正 (EntryExitCalendarService.java)

Service層 (EntryExitCalendarService.java) の requestTimeChange メソッドは、実は前回の実装のままでもおおよそ動作します。

しかし今回の「当日でExitが未入力状態で申請された場合」に、過去の確定退勤時刻（もしあれば）を不用意に消してしまわないよう、より安全に処理を流すための配慮を入れておくと完璧です。

EntryExitCalendarService.java の requestTimeChange メソッド内を以下のように記述してください。

```
/**
 * 業務ロジック2: ユーザーからの打刻修正（編集）申請を処理する
 */
@Transactional
public void requestTimeChange(String accountId, java.time.LocalDate recordDate, String tmpEntry, String tmpExit, String reason) {

    // 1. その日のレコードが既にDBにあるかどうかを検索
    Optional<EntryExitCalendar> existingOpt = calendarRepository
        .findByRegisteredAccountIdAndRecordDate(accountId, recordDate);

    EntryExitCalendar targetRow;

    if (existingOpt.isPresent()) {
        targetRow = existingOpt.get();
    } else {
        // データが1件もない日の場合、新しく行を立ち上げる
        targetRow = new EntryExitCalendar();
    }
}
```

```

        String timePart = java.time.LocalDateTime.now().format(java.time.format.DateTimeFormatter.ofPattern("yyyyMMddHHmmssSSS"));
        String uuidPart = java.util.UUID.randomUUID().toString().substring(32);
        targetRow.setRecordId("REC" + timePart + uuidPart);

        targetRow.setRecordDate(recordDate);
        targetRow.setRegisteredAccountId(accountId);
        targetRow.setEntryTime("-");
        targetRow.setExitTime("-");
    }

    // 2. 申請用の仮保存枠 (tmp_xxx) に値をセット
    targetRow.setTmpEntryTime(tmpEntry);

    // 調整：もし当日の申請で、退勤がまだおこなわれておらず、
    // ユーザーもフォームの退勤欄を空（ハイフン）のまま申請してきた場合は、
    // 既存の確定データ（無ければハイフン）の状態を優しく維持して仮保存します
    if ("-".equals(tmpExit) && !"-".equals(targetRow.getExitTime())) {
        targetRow.setTmpExitTime(targetRow.getExitTime()); // 既に打刻済みの退勤時間があればそれをキープ
    } else {
        targetRow.setTmpExitTime(tmpExit); // 入力された値（新しい時間、またはハイフン）をそのままセット
    }

    // 3. 各種ステータスを「申請中」にセット
    targetRow.setIsTimechangeDemand(1); // 時刻修正申請中フラグを「1:申請中」
    targetRow.setTimechangeStatus(0); // ステータスを「0:未承認」
    targetRow.setReason(reason); // 理由を格納
    targetRow.setAdminComment(""); // 過去の管理者コメントをクリア

    // 4. データベースへ保存
    calendarRepository.save(targetRow);
    System.out.println("=== [Service] DBへの申請ステータス書き込みが正常完了しました ===");
}

```

連動の確認テスト（デバッグ）

コードを修正してSpringBootを再起動し、以下のトラフィックテストを行ってみてください。

- **当日の行のテスト:**

- 本日の行の「編集申請」ボタンを押します。
- 退勤時刻（Exit）を**空欄、または - の状態のまま**にして、備考欄に「出勤打刻エラーの修正」と書いて申請ボタンを押します。
- フロントエンドのバリデーションを通過し、バックエンドでも弾かれることなく、画面上に「申請中 (未承認)」のバッジが綺麗に点灯すれば成功です！

- **過去の行のテスト（安全確認）:**

- 念のため「昨日以前」の行で同じように退勤時刻を空欄（ハイフン）にして申請を試みます。
- フロントまたはバックエンドで「過去の日付の申請には、退勤時刻の入力も必須です」と正しくブロックされるか確認します。